PROGRAMACIÓN II · UNIDAD 1 · CLASE 5

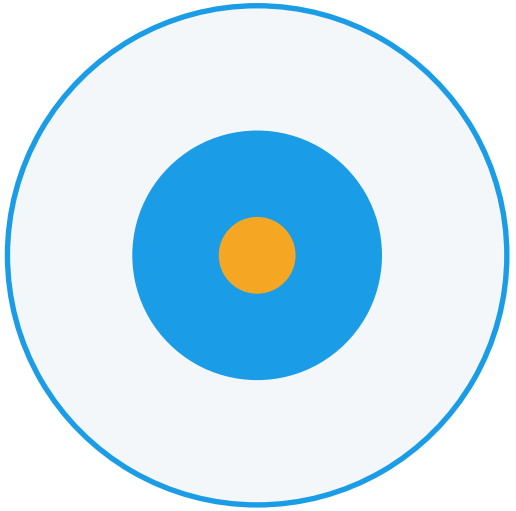
Casteo, sobreescritura, abstracción e interfaces

Bloque 3 · Programación Orientada a Objetos en C#

> *Redefinir el comportamiento por herencia y modelar con clases abstractas e interfaces.*

Objetivo de la clase

Redefinir comportamiento por **herencia** y modelar con **clases abstractas** e **interfaces**.



Qué veremos hoy

1. Upcasting y downcasting
2. Sobreescritura (override / virtual) y base
3. Clase abstracta
4. Composición, agregación e interfaz

1 Upcasting: tratar la derivada como su base

La herencia habilita el **upcasting**: tratar una variable como alguno de sus antecesores. Puede ser implícito o explícito.

casting.cs

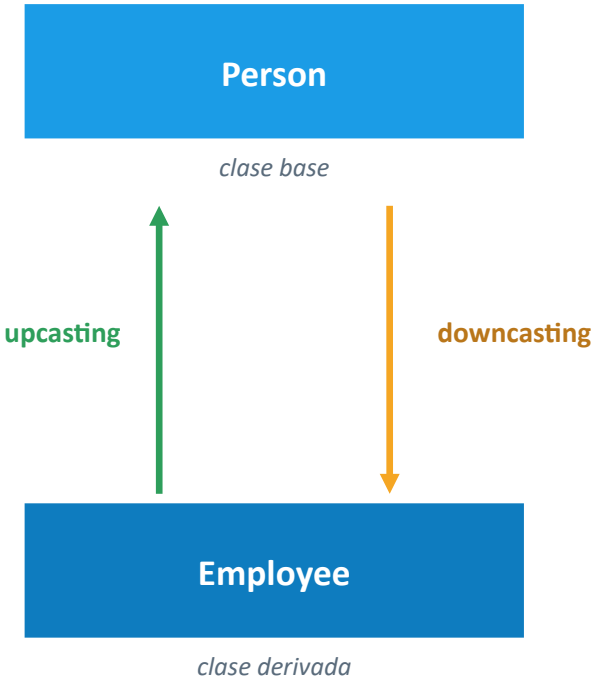
```
// implícito
Person p = new Employee();

// explícito
Employee e = new Employee();
Person p2 = (Person)e;
```

- Habilita el tratamiento genérico: tratar Employee o ProviderEmployee como Person.
- Se pierde el acceso a los miembros propios de la subclase.
- No reasigna memoria: solo referencia al objeto abarcando los miembros de su tipo.

Apunte Unidad 1, pág. 20-22

1 Downcasting: volver al tipo original




```
// Ejemplo 13.2

Person ep = new Employee();      // upcasting
Employee e = (Employee)ep;       // downcasting
```

- Solo se puede castear al mismo tipo de la instancia original.
- Es posible porque, al crear el objeto, se reservó el espacio para todos sus miembros.
- Recupera el acceso a los miembros propios de la subclase.

Apunte Unidad 1, pág. 20-22

2 Sobreescritura: override y virtual

Redefinir el comportamiento de una subclase manteniendo el ***qué*** y cambiando el ***cómo***, respetando la firma. En C#: `virtual` en el padre, `override` en la subclase.

SayHi.cs

```
public class Person
{
    public virtual void SayHi()
    {
        Console.WriteLine($"Hi, I'm {GetFullName()}");
    }
}

public class Employee : Person
{
    public override void SayHi()
    {
        Console.WriteLine($"...my file is {File}");
    }
}
```

- Se reimplementa el método en la subclase respetando la firma.
- El padre lo marca virtual; la subclase, override.

Aun tras un upcasting, se ejecuta la versión definida en la subclase (mecanismo de sobreescritura).

Apunte Unidad 1, pág. 22-24

2 La palabra reservada base

`base` referencia a la clase padre y permite acceder a sus miembros. Tres motivos para sobreescribir:

- Reemplazar el código (cambiar el comportamiento por completo).
- Extender funcionalidad (reutilizar la del padre y agregar).
- Constructores (llamar a uno del padre).

Constructores: si el padre tiene al menos un constructor personalizado, la subclase debe llamar a alguno con `base(...)`.

Employee.cs

```
public class Person
{
    public Person(string name,
                  string lastname) { }

    public virtual void SayHi() { }
}

public class Employee : Person
{
    public Employee(string name,
        string lastname, int file)
        : base(name, lastname)  // padre
    {
        File = file;
    }
}
```

Apunte Unidad 1, pág. 24-27

3 Clase abstracta

- No puede instanciarse.
- Puede tener métodos concretos (con implementación) y abstractos (sin implementación).
- Reúne características necesarias para las subclases, pero no suficientes para tener identidad propia.
- Los métodos abstractos deben implementarse (sobreescribirse) en las subclases.

Shape.cs

```
public abstract class Shape
{
    public abstract double Area();
}

public class Rectangle : Shape
{
    public double Width { get; set; }
    public double Height { get; set; }

    public override double Area()
    {
        return Width * Height;
    }
}
```

Apunte Unidad 1, pág. 27-29

4 Composición y agregación

Relación de dependencia entre un **compuesto** y su **componente**: ocurre cuando una clase tiene un atributo cuyo tipo es otra clase (no un tipo primitivo).

Fuerte **Composición**

El objeto principal no puede existir sin el componente. Ej.: un auto y su motor.

Débil **Agregación**

El objeto principal no requiere necesariamente del componente. Ej.: un auto y su aire acondicionado.

Vehicle.cs

```
public abstract class Vehicle
{
    private Engine engine;                  // fuerte (composición)
    private AirConditioner airConditioner;  // débil (agregación)
    public bool TurnEngineOn() => engine.TurnOn();
}
```

Apunte Unidad 1, pág. 29-32

4 Interfaz: el contrato de comportamiento

Define **solo las firmas** de los métodos que una clase debe implementar: el contrato, incluso entre clases de distinta jerarquía.

ITurnable.cs

```
public interface ITurnable
{
    bool TurnOn();
    bool TurnOff();
}

public class Engine : ITurnable { /* ... */ }
public class AirConditioner
    : ITurnable { /* ... */ }
```

- Se implementa con ":"; varias interfaces se separan con comas.
- Implementar sus métodos es obligatorio.
- Un objeto puede tratarse según cualquier interfaz que implemente.

Programar pensando en interfaces permite sustituir el objeto subyacente por otro que se comporte de la misma manera, sin tocar el resto del código.

Apunte Unidad 1, pág. 33-35

EN SÍNTESIS

- Upcasting: tratar la derivada como su base (pierde miembros propios, no reasigna memoria).
- Downcasting: volver al tipo real de la instancia y recuperar sus miembros.
- Sobreescritura: virtual en el padre, override en la subclase (mismo llamado, distinta respuesta).
- base referencia al padre; en constructores personalizados es obligatoria.
- Clase abstracta: no instanciable; sus métodos abstractos se implementan en las subclases.
- Composición/agregación (fuerte vs. débil) e interfaz como contrato; aportan al polimorfismo.

Próxima clase (práctica): casteo, override, constructores encadenados y una jerarquía con clases abstractas e interfaces.